Testing RAG and AI agents: interview questions, answered with this project

Questions about testing retrieval, grounding and agents that come up in SDET and QA lead interviews on AI products. Where the project shows the idea, the answer ends with an **In the project** pointer. Links go to github.com/niranjankagri/rag-agent-evals.

Versions: Python 3.13, pytest 9.

Contents

1. Evals and AI testing basics (Q1-Q12)
2. Testing RAG (Q13-Q30)
3. Testing agents (Q31-Q45)
4. Security, gates and strategy (Q46-Q60)

1. Evals and AI testing basics

Q1. How is testing an AI feature different from ordinary testing?

The output can vary, several answers can be right, and a change of model or prompt can move behaviour with no code change. So you measure over a set of cases, not one example.

Q2. What is an eval?

A repeatable measurement of a system on a versioned set of cases, with scoring and a limit.

In the project: four suites in `datasets/`, scored by `src/evals/`, limits in `configs/quality_gate.toml`.

Q3. Eval or test: what is the difference?

A test passes or fails on one behaviour. An eval gives a score across many cases. I use both: tests for rules that must always hold, evals for quality.

In the project: a refund limit is a test, retrieval quality is an eval.

Q4. What makes a good eval case?

An input, the evidence it needs, the expected behaviour and how it's graded. It should be reviewed like code.

In the project: a line in `rag_cases.jsonl` holds the question, the relevant section and the fact it must contain.

Q5. Why version the dataset?

A score only means something against a known set of cases. If the cases change quietly, two scores can't be compared.

Q6. Rule-based or model-based grading?

Rules where the answer can be checked exactly: they're free, fast and repeatable. A model judge for what rules can't see, such as tone. Use the cheapest grader that does the job.

In the project: rules only.

Q7. How do you trust an LLM judge?

Calibrate it: compare its scores with human labels on a sample, and track the agreement. An uncalibrated judge is an opinion.

Q8. How do you handle answers that vary between runs?

Run each case several times, report a pass rate, and check meaning and facts, not exact wording.

In the project: the system is deterministic, and `TC-141` proves it. My llm-feature-testing repository covers repeated runs.

Q9. What should you do when a score is 100 %?

Be suspicious. Check whether the cases are too easy, or were used to tune the system.

In the project: the RAG suite is 100 %, and the README says why that proves little. Robustness is 53 %.

Q10. What is a held-out set?

Cases kept away from tuning, so they measure generalisation. Without one, you measure how well you fitted your own questions.

In the project: there isn't one, and the design document lists that as a limit.

Q11. How do you prove a grader works?

Give it bad answers on purpose and check that it fails them.

In the project: `TC-704` plants a changed fact, an invented sentence, a missing citation and a made-up citation.

Q12. What belongs in a trace?

Inputs, retrieved evidence, tool calls, results, timing and versions, with personal data removed.

In the project: `reports/traces.jsonl`, redacted, one line per case.

2. Testing RAG

Q13. What is RAG?

Retrieval-augmented generation: find relevant documents first, then answer from them.

Q14. What are the two halves you test?

Retrieval and generation. Test them apart, or you won't know which one failed.

In the project: recall and rank measure retrieval; correctness and grounding measure the answer.

Q15. What is recall at k?

The share of the relevant documents found in the first k results.

In the project: `recall_at_k`, tested by `TC-701`.

Q16. What is mean reciprocal rank?

The average of 1 divided by the position of the first relevant result. It rewards finding it early.

Q17. Precision or recall for RAG?

Recall first: if the evidence isn't retrieved, the answer can't be right. Precision matters next, because noise can mislead the generator.

Q18. What is groundedness?

Every claim in the answer is supported by the retrieved evidence.

In the project: `is_grounded` requires each sentence to appear in a cited section.

Q19. Grounded and correct: can one be true without the other?

Yes. An answer can quote the wrong section faithfully. It's grounded, and wrong.

In the project: `TC-202` is exactly that: a real sentence from the wrong section.

Q20. How do you test citations?

Check that each citation exists, that it supports the claim, and that a made-up one fails.

Q21. How do you test "I don't know"?

Include questions the knowledge base can't answer, and require no answer.

In the project: `TC-131` to `TC-135`.

Q22. Why count a wrong answer apart from a miss?

A miss sends the user to a person. A confident wrong answer misleads them. They aren't the same risk.

In the project: `no_wrong_answer` is 0.867 where `answer_correct` is 0.533.

Q23. How do you decide when to abstain?

A threshold on the evidence, tuned on cases, then tested on both sides.

In the project: `min_score` and `min_terms`, added after "number" matched a phone question to lost parcel claims.

Q24. How do you test conflicting sources?

Put an old and a new rule in the knowledge base and require the new one.

In the project: `N1` replaces `R3`, and `TC-221` and `TC-241` check it.

Q25. What is chunking, and why test it?

Splitting documents for retrieval. Bad splits separate a fact from its context.

In the project: one section is one chunk. `TC-603` checks the parsing.

Q26. How do you test paraphrases?

Ask the same thing in other words, and expect the same fact.

In the project: the robustness suite. It's where keyword search breaks.

Q27. Keyword or embedding search?

Keywords are exact and explainable, and miss synonyms. Embeddings handle meaning, and can miss exact codes. Many systems combine both. The eval should decide, not fashion.

Q28. What would you do about the 53 %?

Add embedding or hybrid retrieval, then rerun the same suite. The cases are ready, and strict `xfail` would flag each fix.

Q29. What is strict xfail for?

It marks a known failure, and fails the run if the test starts to pass, so the list can't go stale.

In the project: `KNOWN_MISSES`.

Q30. How do you test the knowledge base itself?

Check its format, its access labels, and for replaced or injected content, when it's loaded.

In the project: `knowledge.py`, with `TC-322` and `TC-603`.

3. Testing agents

Q31. What makes testing agents harder?

They act. A wrong step can change data, and the path can differ between runs. So you test the path, the outcome and the limits.

Q32. What do you check for a task?

The tools called, the arguments, the final status, and the state afterwards.

In the project: `agent_cases.jsonl` holds all four.

Q33. Why check the final state?

An agent can report success without doing it. The data is the truth.

In the project: `state_matches` reads the orders, not the agent's message.

Q34. Where do guardrails belong?

Outside the model, in code. A prompt is a request; code is a rule.

In the project: every check is in `agent.py`, not in the planner.

Q35. How do you test guardrails without a model?

Script the bad behaviour, then check that the code stops it.

In the project: `ScriptedPlanner` plays a model asking for forbidden things.

Q36. How do you test tool arguments?

Missing, extra, wrong type, and values that look right, such as `True` where a number is expected.

In the project: `TC-502`. In Python, `True` is an integer, so it needs its own check.

Q37. How do you test permissions?

Try each action as a user who shouldn't have it, for reading as well as writing.

In the project: `TC-503` and `TC-504`.

Q38. Should a run continue after a refusal?

Not after a permission failure. Letting it try other things is how data leaks.

In the project: `TC-505`.

Q39. How do you test stopping limits?

A plan that repeats, and a plan that never repeats but never ends. They need different checks.

In the project: `TC-509` and `TC-510`.

Q40. Why not retry every failed tool call?

A timeout doesn't say a change failed. Retrying a refund could pay twice.

In the project: `TC-512` loses the answer after the money moved, and checks that only one refund happened.

Q41. How do you stop a limit being split into small pieces?

Count the running total, not each call.

In the project: `TC-507` tries 30 plus 25 against a limit of 50.

Q42. When should an agent hand over to a person?

When an action is above its authority, and the hand-over should be tested like any other outcome.

In the project: `TC-403` and `TC-421`.

Q43. How do you test text inside a tool's answer?

Put an instruction in the data and check that nothing follows it. This matters more with a real model.

In the project: `TC-513`. The rule planner ignores all such text, so this proves little, and the README says so.

Q44. What is the difference between an agent's message and its outcome?

The message is what it says. The outcome is the status and the state. Test the outcome.

Q45. How would you test this with a real model?

Keep the cases, graders and guardrails, replace the planner, run each case several times, and report pass rates.

4. Security, gates and strategy

Q46. What is prompt injection?

Text that tries to override the system's instructions. Direct injection comes from the user; indirect comes through documents or tool results.

Q47. How do you test indirect injection in RAG?

Plant an instruction in a document, and check that it never reaches an answer.

In the project: `U1` in `community-tips.md`, with `TC-311` and `TC-322`.

Q48. Is a word list enough to stop injection?

No. It stops the obvious. Real defence is layered: least privilege, guardrails in code, and treating retrieved text as data.

Q49. How do you test access boundaries in RAG?

Ask as a user who shouldn't see a document, and check that it isn't even retrieved.

In the project: `TC-321`.

Q50. Why filter before ranking, not after?

Anything retrieved can reach an answer. A later filter is one bug away from a leak.

Q51. How do you test for data leaks?

Ask for other people's data, and check answers and logs for what must not appear.

In the project: `must_not_contain` in the security cases, and `TC-722` for traces.

Q52. What should a quality gate check?

Quality limits, and that the measurement really happened.

In the project: a case count and a missing-metric rule, as well as the limits.

Q53. How do you set a limit when a score is poor?

Use a baseline just under today's value, to stop things getting worse, and say clearly that it isn't a target.

In the project: the comment in `quality_gate.toml`.

Q54. When should evals run?

After any change to a prompt, model, tool, retrieval or data, not just code.

Q55. How do you know your own gate works?

Break the system on purpose and watch it fail.

In the project: 20 bugs. That found a real gap: with quarantine off, the gate passed until `TC-311` was added.

Q56. Tests or gate: which caught more?

The tests caught all 20, and the gate 9. The gate only sees behaviour on its cases, so narrow rules belong in tests.

Q57. What did the first eval run find?

Answers based on one shared word, a weak stemmer, and a bug in my own grader, which scored polite refusals as ungrounded.

Q58. What do you measure in production that you can't measure here?

Real questions, cost, latency under load, and drift.

Q59. What metrics would you report to a manager?

Task success, wrong-answer rate, and the worst failures with examples. One average hides what matters.

Q60. What does this project not prove?

How a real model behaves. There's no model here, so it can't invent text or be persuaded. It proves the method, the checks and the guardrails.